Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Bearbeitungszeitraum: von 20. Januar 2026
bis 20. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: June 16, 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Aufgabensteller, Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in (Firma/Behörde/Hochschule):	OTH Amberg-Weiden
Betreuer in Firma/Behörde:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum: 20. Januar 2026	Abgabedatum: 20. Juni 2026

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

---

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Abstract:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Keywords: LaTeX, Textsatz, Formeln, Graphiken

Contents

Acronyms

IL Imitation Learning

LfV Learning from Videos

LLM Large Language Model

RL Reinforcement Learning

VLA Vision-Language-Action

Mathematical Notation in Formulas

Capital Letters

Capital Letters represent *random variables*.
Examples:

- S: Represents the set of all states in which the robot can be.
- A: Represents the set of all possible actions the robot can perform.

Lowercase Letters

Lowercase letters represent *specific values* of *random variables*.
Examples:

- $s \in S$: Stands for a specific robot state.
- $a \in A$: Stands for a specific action that the robot performs.

Timestep t. A lowercase letter is usually followed by a time index t, representing the specific action taken by the robot at time step t. Examples:

- $s_t \in S$: Stands for a specific robot state at time step t.
- $a_t \in A$: Stands for a specific action that the robot performs at time step t.
- In some cases the time index is omitted for simplicity, e.g. a instead of a_t.
- In the literature, the next state or action is sometimes denoted as s' or a' instead of s_{t+1} or a_{t+1}.

Capital Letters in Bold

Capital letters in bold represent matrices.
For example:

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix} \tag{1}$$

Lowercase Letters in Bold

Lowercase letters in bold represent vectors.
For example:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \tag{2}$$

Chapter 1

Experiments on a Real Robot

The second part of this work will focus on the practical implementation of an open-source Imitation Learning (IL) approache on a real robotic system. The goal is to gain hands-on experience with the challenges and opportunities how to run a robot model on a real robot. This includes finding out how to connect a robot with the PC, how to control the robot from the PC and what additional code is needed to adapt a robot model to run on our specific robot. The idea was to implement a easy-to use basic pipeline to communicate with the robot, with an framework where only the robot model must be replaced. This includes also a camera setup with camera calibration to capture the robot's environment to provide the necessary visual input for the robot model. The whole pipeline should be designed to be modular and extensible, allowing for easy integration of different robot models with different software dependencies.

Overview of the steps to setup and run a robot model on a robot:

1. Setup Connection between PC and Robot.
2. Setup the control protocols, for controlling the Robot via PC (for example within a Python script [1]). There exists two different control protocols:
   - RTDE: Direct control Interface provided by Universal Robots [2].
   - ROS 2: A popular high-level open-source framework for controlling different robots with the same Interface [3].
3. Camera Setup and Calibration.
4. Implementing the robot model on the PC and adapt it to the UR5e.
5. Testing the robot model on the real robot and evaluate the performance.

Robot Hardware

For experiments, the UR5e Robot Arm [**UR5e_1**] from Universal Robots will be used. It is a collaborative robot (cobot) designed for safe human-robot interaction. The UR5e provides a versatile and user-friendly 12 inch touchscreen interface for controlling and

static programming the robot. The Robotiq Gripper 2F-140 was used as the gripper and mounted on the UR5e with a payload of 2.5 kg and a stroke of 10 to 125 Newton [4]. In Table 1.1 some specifications of the Robot are listed.

Robot: UR5e Robot Arm from Universal Robots	
Degrees of freedom	6 rotating joints
Joints speed	180°/s
Payload	5 kg
Reach	850 mm
Weight	12 kg
Materials	Powder coated steel

Table 1.1: Specifications of the UR5e Robot Arm. Source: [5]

1.1 Setup Connection from PC to Robot

Many modern robot learning models require substantial computational resources for image processing, perception, and action generation. In particular, many open-source Vision-Language-Action (VLA) models rely on high-performance GPUs, such as an NVIDIA RTX 4090, to achieve real-time or near real-time inference.

Due to these computational requirements, the robot model must be deployed on an external workstation, rather than directly on the UR5e robot, whose onboard hardware is not designed for executing large-scale deep learning models. In addition, some extra software is needed for running a robot model and communicating with the robot control interface, which can be easily installed on a PC. Consequently, a communication connection between the workstation and the robot had to be established, which enables the external computer to execute the robot model, send motion commands to the robot or receive sensor and state information from the robot required for closed-loop control.

Setup Guide. To facilitate this process, a detailed setup guide was created that documents the individual steps required to establish a reliable connection between the workstation and the UR5e robot. The guide covers the configuration of network settings and communication ports on both devices, procedures for verifying network connectivity, the activation of Remote Control mode and required services on the robot, as well as the configuration of the External Control functionality. This includes the creation of an External Control program that must be executed on the robot before control commands can be transmitted from the workstation.

1.2 Setup Control Protocols

1.2.1 Robot Control Protocol: ROS 2

ROS 2 (Robot Operating System 2) [3] is an open-source robotics middleware framework used for building modular and distributed robotic systems. It organizes software into independent components called nodes, where each node performs a specific function such as perception, control, or planning. These nodes communicate with each other using standardized interfaces, including topics, services, and actions. ROS 2 provides the communication infrastructure that enables reliable data exchange between different parts of a robotic system. In this way, it acts as the "plumbing" layer that connects sensing, computation, and actuation in robotics applications. A key advantage of ROS 2 is that it provides a standardized interface, allowing the same software components to be reused across different robotic platforms without requiring changes to the core code, as long as the robots support the same ROS 2 communication interfaces.

The complete ROS 2 implementation developed during this work is publicly available in the following repository:

`https://github.com/sweidner2001/RobotUR5e_ROS2`

The repository contains scripts and configuration files required to establish communication with the UR5e robot and to launch the corresponding ROS 2 nodes. These scripts automate the initialization process and provide a easy-to-use setup for robot control, visualization, and simulation. The ROS 2 environment integrates several commonly used robotics tools:

- **RViz:** RViz [6] is a 3D real-time visualization tool for ROS, see Figure 1.1. It provides a view of the robot model, sensors, and other information in a 3D space. It can be used to display sensor data from robots, such as point clouds, camera images, robot models and robot trajectories. While RViz can be used in conjunction with Gazebo, it is not a simulation environment of itself.
- **MoveIt:** MoveIt [7] is a motion planning framework for robotic manipulators that computes collision-free trajectory generation and collision checking between start and target poses, see Figure 1.2. It can be used for both simulated and real robotic systems.
- **Gazebo:** Gazebo [8] is a physics-based simulation environment commonly used in conjunction with ROS 2 and RViz. It enables realistic simulation of robot dynamics (mass, inertia, gravity), sensors (cameras, lidar, force sensors) and the environment (collisions, friction). Gazebo provided a virtual testing environment for validating robot configurations and control algorithms before deployment on the physical UR5e robot.

In addition to the software implementation, comprehensive documentation was created to support the deployment of a robot model via ROS 2. The documentation describes the procedures required to establish communication with the robot via ROS 2, set

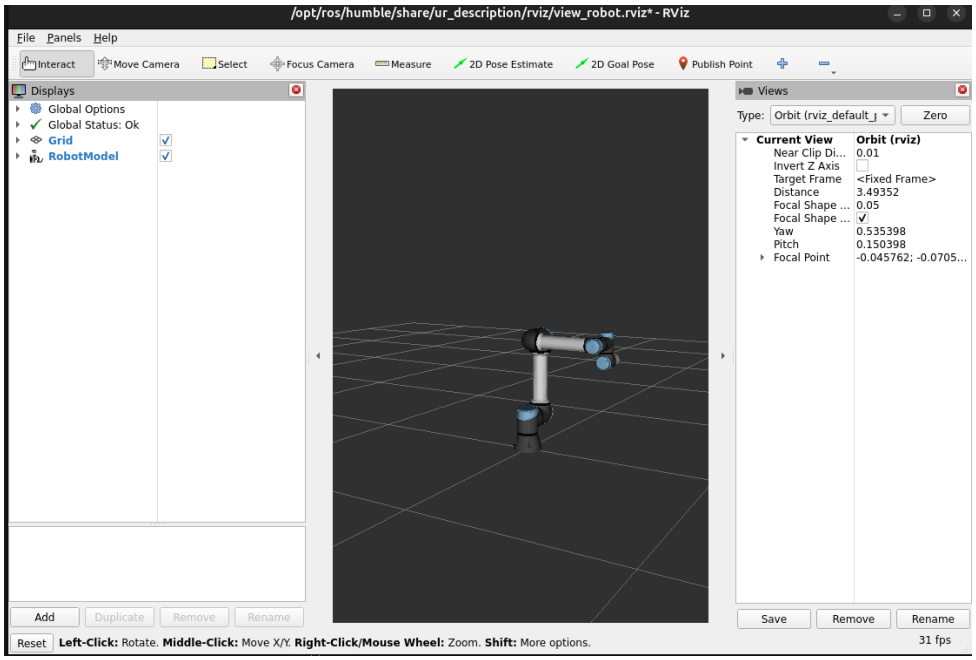

Figure 1.1: RViz is a 3D real-time visualization tool used in ROS 2.

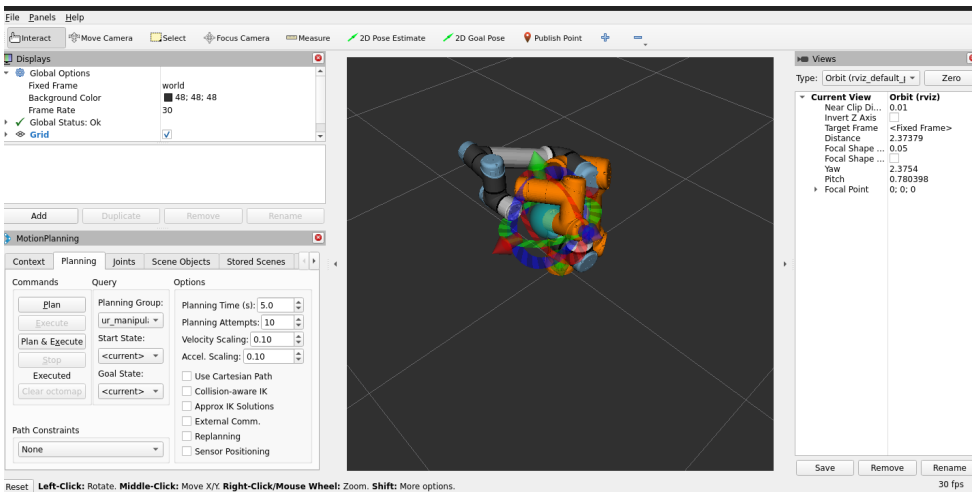

Figure 1.2: MoveIt is a motion planning framework and here used together with RViz. The orange robot represent the target pose of the robot and within the RViz you can simulate the trajectory, which is planned by MoveIt.

the UR5e robot in External Control mode, ROS 2 commands to control the robot and basic ROS 2 commands for troubleshooting. Furthermore, it explains the usage of the provided scripts and workflows for controlling the robot within both simulated and real-world environments.

Setup

To simplify the development and deployment process, the ROS 2 project was implemented using a Dev Container environment. This approach provides an isolated and reproducible software environment, eliminating the need to install ROS 2, robot drivers, and additional dependencies directly on the host system. As a result, all developers can work within a consistent environment while avoiding dependency conflicts and configuration issues. The Dev Container provides a fully configured ROS 2 Humble workspace tailored for Universal Robots development. It includes the required communication interfaces for the UR5e robot, as well as commonly used robotics tools such as Gazebo for simulation, MoveIt for motion planning, and RViz for visualization.

The Dev Container setup follows a layered workflow consisting of four main components:

1. **devcontainer.json:** Defines the configuration for the development container, including user settings, mounted directories, environment variables, Visual Studio Code extensions, and post-creation commands. It serves as the entry point for building and configuring the container environment.
2. **Dockerfile:** Defines the base development environment and installs all required software packages, including ROS 2 Humble, the Universal Robots driver, Gazebo, RViz, and the robot simulation workspace. During the image build process, the script `setup_workspace_additional.sh` is executed.
3. **setup_workspace_additional.sh:** This script contains permanent configuration steps that become part of the Docker image and are therefore available in every container instance created from the image. You can add easily your additional setup here to extend the docker image without editing the base Dockerfile.
4. **setup_workspace_temp.sh:** After the Docker image has been built successfully, the `devcontainer.json` configuration executes the script `setup_workspace_temp.sh` through the `postCreateCommand`. In contrast to the permanent setup script, this script is executed each time a new container instance is created. It is intended for temporary workspace configuration, experimental package installations, and project-specific modifications that should not require rebuilding the entire Docker image. This separation between permanent and temporary configuration steps provides a flexible and fast development workflow while maintaining a stable and reproducible base environment.

Script Name	Description
`launch_std.sh`	Launches the UR robot driver to connect to a real UR robot. This allows for direct control of the robot through ROS 2 topics and services. If the command is successful, RViz should start and shows the robot in its current pose. If the robot is not visible, there is a connection error.
`launch_RViz_MoveIt.sh`	Launches the UR robot driver together with MoveIt and RViz. This allows controlling the real robot via the MoveIt planning interface in RViz.
`sim_launch_RViz _Gazebo.sh`	Launches the UR5e simulation in Gazebo with RViz. The robot is simulated and can be visualized via RViz.
`sim_launch_RViz _MoveIt_Gazebo.sh`	Launches the UR5e simulation in Gazebo with MoveIt and RViz. In addition to the simulation, MoveIt is loaded, enabling motion planning within the simulation.

Table 1.2: Launch Scripts for UR5e Robot Control

Provided Scripts

Gripper Control

To control the gripper, additional ROS 2 packages and interfaces are required, as the gripper is not directly controlled through the main robot driver. Due to missing instructions, how to control the gripper directly with a ROS 2 Node, the gripper wasn't finished implemented and is not functional yet. Some additional specific urdf.xacro files are needed, to connect the gripper to the robot model in Rviz. However, the ROS 2 setup provides the necessary infrastructure to integrate gripper control in the future.

Summary

In summary, the ROS 2 implementation provides a comprehensive framework for controlling the UR5e robot arm from an external workstation, but there was some difficultness with the gripper control, which is not yet implemented. For this reason, the decision was made to prioritise the implementation of the RTDE interface provided by Universal Robots, as ready-made Python packages [1] were already available for this purpose. The aim was to implement an initial working robot model as quickly as possible.

1.2.2 Robot Control Protocol: RTDE

RTDE (Real-Time Data Exchange) [2] is a direct communication interface provided by Universal Robots for exchanging data between an external computer and the robot controller. It enables fast, low-latency transmission of sensor data and control commands. Compared to ROS 2, RTDE operates closer to the hardware and therefore

provides higher update rates. This makes it suitable for real-time robot applications in the Universal Robots universe, where fast and reliable data exchange is required. The advantage is that RTDE is easy to set up. However, the disadvantage is that the interface must be adapted when switching to a different robot.

The complete RTDE implementation developed during this work is publicly available in the following repository:

`https://github.com/sweidner2001/RobotUR5e_RTDE`

Provided Scripts

Some test scripts are provided, for example a static pick-and-place demo with three cubes, which are stacked on top and another script for the reverse process. The scripts demonstrate how to use the RTDE interface to control the robot's movement and gripper, as well as how to read sensor data from the robot. The provided code serves as a starting point for implementing more complex robot control algorithms and can be easily extended to include additional functionality or to adapt to different robotic tasks.

Setup

To simplify development and deployment, the RTDE project was implemented in a dedicated Python virtual `venv`-environment [9]. This approach provides an isolated and reproducible setup that can be executed on any system with Python installed, while avoiding dependency conflicts with globally installed packages.

The environment can be easily set up by executing the `setup.sh` script, which automatically creates the virtual environment and installs all required dependencies for working with the RTDE interface. In addition, the script configures the project structure such that Python modules can be imported using absolute paths relative to the project root. All project scripts on every location in the project structure can be conveniently executed from within Visual Studio Code using the "run button" functionality, with all imports resolving correctly once the environment is activated.

Provided Scripts

The project includes several example scripts for testing and demonstration purposes. One example implements a static pick-and-place scenario in which three cubes are stacked in a predefined order, while another script performs the reverse operation. These examples demonstrate how the RTDE interface can be used to control robot motion and gripper actions, as well as how to access and process sensor data from the robot. The provided implementations serve as a starting point for more advanced robot control strategies. They are designed to be easily extensible, allowing additional functionality to be integrated or adapted to different manipulation tasks.

1.3 Summary

Overall, the robot setup required a significant amount of time, in particular the ROS 2 implementation, which ultimately remained incomplete and did not always operate reliably. However, by using the RTDE interface, it was still possible to quickly run a simple static pick-and-place demonstration on the UR5e robot.

Due to limited time, no full robot learning model could be deployed on the UR5e within the scope of this thesis. Nevertheless, initial considerations were made regarding the placement of cameras around and on the robot. A suitable 3D model for a camera mount on the robot was also identified for potential 3D printing.

In addition, an overview of ten promising open-source robot learning models was created, which are expected to be implemented with low integration effort in future works.

Chapter 2

Summary, Conclusion & Outlook

2.1 Summary

This thesis examined imitation learning as a promising approach for enabling robots to acquire complex skills from demonstrations and move toward the long-term goal of general-purpose robotic intelligence. Large Language Models (LLMs) have demonstrated that training powerful architectures on vast and diverse datasets can lead to remarkable generalization capabilities, enabling systems to solve tasks for which they were never explicitly trained. A central goal of imitation learning is to bring similar capabilities to robotic systems by leveraging the enormous amount of video data available on the internet.

Chapter **??** introduced the fundamental concepts of imitation learning and distinguished between two major paradigms: Behavior Cloning and Learning from Videos (LfV). While Behavior Cloning relies on demonstrations containing explicit action labels, Learning from Videos seeks to exploit the vast amount of action-free video data available online, offering a scalable alternative for robot learning.

Chapter **??** discussed the overarching objective of robotics research: the development of generalist robots capable of performing a wide range of tasks in unstructured and previously unseen real-world environments. A major obstacle to this vision is the robotics data bottleneck, which arises from the limited availability of large, diverse, and high-quality robotic datasets. Internet videos represent a potential solution due to their abundance and diversity; however, their use introduces additional challenges related to the transfer of visual knowledge to robotic systems.

A prerequisite for effective human-robot interaction is the ability to communicate goals and intentions to robotic agents. Therefore, chapter **??** examined how robots can be instructed and guided to perform desired tasks.

Chapter **??** focused on the central research questions of Learning from Videos: how relevant knowledge can be extracted from internet videos and how this knowledge can subsequently be transferred to robotic systems for policy learning and control. Video Foundation Models were introduced as a powerful tool for extracting semantic

and behavioral information from large-scale video data. Two major challenges were identified in the transfer process: the absence of action labels and the distribution shift between human videos and robotic observations. To address these challenges, alternative action representations are employed to relabel action-free video data, enabling robots to learn from demonstrations that do not contain explicit control signals.

Chapter **??** explored the role of robust video understanding in IL. Learning manipulation skills from video requires more than simple action recognition: (1) It demands meaningful representations of the environment, (2) an understanding of object affordances and human-object interactions, (3) recognition of human activities, and (4) accurate modeling of fine-grained hand movements. These capabilities form the basis for a general-purpose robot that can interpret human demonstrations.

Chapter **??** focused on approaches for learning coherent robot models from video data. (1) The chapter first introduced foundational perception and representation learning methods and techniques. (2) It then discussed both RL-based and imitation learning approaches, highlighting their respective strengths and limitations. (3) Hybrid methods, which combine the structured guidance and sample efficiency of imitation learning with the exploration and optimization capabilities of reinforcement learning. Special attention was given to the intersection of Learning from Videos (LfV) and Reinforcement Learning (RL), demonstrating how knowledge extracted from videos can be used to improve policy learning and enhance the sample efficiency of RL algorithms. (4) Last but not least, the chapter presented multi-modal learning approaches and Vision-Language-Action (VLA) models and the three Architecturial paradigms, including sensorimotor, world-model-based, and affordance-based architectures. VLA models represent currently the state of the art in video-based robot learning.

Chapter **??** examined the major challenges and future research directions in IL and video-based robotic learning. Despite significant advances in recent years, the development of robust and generalizable robotic systems remains constrained by several fundamental limitations, including data availability, computational scalability, domain transfer, and efficient policy learning. These challenges highlight the gap between current research systems and the long-term goal of general-purpose robotic intelligence. At the same time, emerging approaches such as diffusion-based policies, world models, and causally-aware learning systems offer promising opportunities for overcoming these limitations and improving the adaptability, reasoning capabilities, and generalization performance of future robotic agents.

Finally in Chapter **??**, selected IL approaches deployed on real robotic systems were examined to provide practical insight into the theoretical concepts discussed throughout the thesis. These examples illustrated how IL can enable robots to acquire complex manipulation and control skills directly from video demonstrations.

2.2 Outlook

Implementing and testing a open-source Robot model on our UR5e Robot will be the next step. This will provide practical insights into the challenges of deploying IL approaches on real robotic systems and allow for the evaluation of different methods in a real-world setting. Implementing the communication to the robot with RTDE will be the first step, then it can be done with ROS 2.

There exists many different Architectures which could be used: Timesformer, Q-Former, Use of Spike Neural Networks, Graph Neural Networks and Causal Reasoning. There exist many different Architectures of VLA and Multi-Modal Learning / Models. Action Chunking Transformer which is maybe better than BC: ACT is preferred when you want a robust, smooth, long-horizon policy distilled from demonstrations, whereas standard BC may fail if the task requires multi-step reasoning. - Conditional Behavior Transformers (C-BeT) - Forschungsrichtung: Affordance grounding for robot interaction - conditional variational autoencoder (C-VAE)

Other Research directions for next works are: The field of Robotics provides a large field of research. Further work could examine the available Datasets and categories. Examine more implemented successful Architectures to become a better understanding, what is possible. There is a lot of Research of VLA Models to robotics. Meta-Learning Approaches could be examined. The integration of Causal Reasoning and World Models could be further explored.

RL techniques and how they can combined with IL approaches, for example through finetuning to increase task success.

There is no work comprehensively comparing the utility of different action representations for LfV.

LfV Policy Methods: - **Future Directions:** - Jointly pretraining **multi-modal foundation models** (on both video and robot data) remains underexplored. - Use **spatio-temporal encoders** and large internet datasets. - representations from video foundation models Methods that leverage representations $\hat{A}$ will be limited by the scalability of their $\hat{A}$ (discussed in Section 4.2.1). An open question is whether $\hat{A}$'s are best utilised explicitly in a hierarchical setup, or if they are most useful for representation transfer. In hierarchical setups, where the policy is defined as $\pi(a_t|\pi_{alt}(\hat{a}_t|o_t), o_t)$, we should explore the extent to which either π_{alt} or π is the performance bottleneck.

Dynamics Models: scalable alternative-action techniques will be useful for conditioning foundational video predictors. - Exploring **hierarchical world/dynamics models** (higher levels learned from video, lower levels from robot data) is a promising avenue - An underexplored direction is to combine standard analytic simulation with video prediction simulation

- **Category: Visual Question Answering (VQA):** - Ask video-to-text models questions like “Did the robot complete the task?” —> reward = task completion or score progress (Dense Reward) - VQA only used with images as input, but not with video-based VQA Following synthetic data trends in other fields, future work may seek to use

foundational video predictors to generate synthetic video rollouts, or augmentand improve the coverage of existing data via video editing

Reward Functions: Underexplored direction: is to use LfV rewards to augment reward labels in offline RL data.

- **Safety:** LfV reward may useful for detecting **safety-related metrics** - **RL-AI-F:** Extracting shaped reward functions from video-to-text models via **RL from AI feedback (RL-AI-F) methods** is an avenue for future work.

- **Pretrained Foundation Models:** The most promising approaches - including **video-language models** and **video predictors** as reward functions - **Integration:** LfV reward functions could be combined with other LfV KMs,

Value Functions for example, by finetuning an LfV policy through online RL using LfV rewards. TD-learning of value functions from large-scale human video is seen as a promising direction for real-world robotics

Scarce research: few works on value learning from video

Monolithic vs Compositional Models —> compare Approaches - video generation model to hierarchically condition an action-prediction —> example compositional LfV approach - Any-to-any sequence modelling —> example monolithic LfV approach - compositional approaches are more computationally efficient, more data-efficient, and can obtain improved generalization - monolithic models may benefit from positive transfer between the different data modalities and tasks they handle, and can be optimized end-to-end.

Recovering action information from video

1. Explicitly compare the pros and cons of different action representations, and measure the extent to which each can help obtain the benefits of LfV (see Section 3.3). 2. Develop improved, novel action representations. Perhaps by combining the benefits of existing options [310].

Benchmark - Improved benchmarks that quantitatively measure LfV benefits (Section 5.2.2) will facilitate these efforts

We need metrics that capture not only task success but also generalization, robustness, and sim-to-real performance.

- We consider task goals to be described by contact states. - Dadurch werden auf natürliche Weise Mehrdeutigkeiten vermieden, die bei der Betrachtung räumlicher Beziehungen auftreten - **Das Erschließen menschlicher Absichten bei gleichzeitiger Beseitigung der inhärenten Mehrdeutigkeiten in Videos ist eine zukünftige Richtung, die es zu erforschen gilt.** - We have also assumed that demonstration videos must be captured with a stationary camera. In reality, most videos in everyday scenarios contain a moving camera. - Daher besteht eine vielversprechende zukünftige Richtung darin, zu untersuchen, wie sich ein Modell erstellen lässt, das dynamische Szenen aus einer bewegten Kamera rekonstruieren kann, wobei das gewünschte Modell die Geometrie sowohl statischer als auch bewegter Objekte in den Szenen schätzen kann

und gleichzeitig sicherstellt, dass der Maßstab der rekonstruierten Szenen und Objekte der realen Welt entspricht.

Chapter 3

Imitation Learning fundamentals

Was sollte noch erklärt werden?

Online Learning / Offline Learning

Multi-Modal und Foundation Models

Bibliography

[1] "Welcome to Python.org", Python.org. (06/10/2026), [Online]. Available: https://www.python.org/ (visited on 06/16/2026).

[2] "Real-Time Data Exchange." (), [Online]. Available: https://www.universal-robots.com/developer/communication-protocol/rtde/ (visited on 06/16/2026).

[3] "ROS 2 Documentation — ROS 2 Documentation: Rolling documentation." (), [Online]. Available: https://docs.ros.org/en/rolling/index.html (visited on 06/16/2026).

[4] "Adaptive Grippers | Robotiq". (), [Online]. Available: https://robotiq.com/products/adaptive-grippers (visited on 06/16/2026).

[5] *Technical specification UR5e*. [Online]. Available: https://www.universal-robots.com/manuals/EN/TechSheets/UR5e_techsheet_pdf_online/UR5e_techsheet_en.pdf.

[6] "Rviz: Main Page." (), [Online]. Available: https://docs.ros.org/en/lunar/api/rviz/html/c++/index.html (visited on 06/16/2026).

[7] "MoveIt 2 Documentation — MoveIt Documentation: Rolling documentation." (), [Online]. Available: https://moveit.picknik.ai/main/index.html (visited on 06/16/2026).

[8] "Gazebo." (), [Online]. Available: https://gazebosim.org/home (visited on 06/16/2026).

[9] "Venv — Creation of virtual environments", Python documentation. (), [Online]. Available: https://docs.python.org/3/library/venv.html (visited on 06/16/2026).

List of Figures

List of Tables